Lab 1: Classification with Large Language Models

Comparing In-Context Learning and QLoRA Fine-tuning

Peng CHEN

Yiwei ZUO

Master 2 Computational Linguistics

Course: Current Topics in NLP and Society

Prof. Marie Candito

10.2026

Contents

1 Introduction **2**

2 Dataset and Experimental Protocol **2**

2.1 Dataset and classification task . . . 2
2.2 Data splits . . . 2
2.3 Model and experimental setup . . . 2
2.4 Evaluation Metrics . . . 3

3 In-Context Learning **3**

3.1 Prompting and inference . . . 4
3.1.1 Prompt design . . . 4
3.1.2 Prediction and label extraction . . . 4
3.2 Development experiments . . . 5
3.2.1 Experimental design and baselines . . . 5
3.2.2 Initial results: 0, 3, and 6 shots . . . 5
3.2.3 Follow-up experiment: 12 shots . . . 6
3.2.4 Optional Experiment: Generation vs. Label Log-Probability . . . 6
3.3 Final Configuration Selection . . . 7
3.4 Final Evaluation . . . 7
3.4.1 Results on `rand-test` and `freq-test` . . . 7
3.4.2 Error Analysis . . . 8

4 QLoRA Fine-tuning **12**

4.1 Method and Training Setup . . . 12
4.2 Experimental Design . . . 13
4.2.1 RQ1: Does Including the Lemma Help? . . . 13
4.2.2 RQ2: Does Extending LoRA to the FFN Help? . . . 14
4.3 Training Results . . . 14
4.4 Results on `rand-dev` . . . 14
4.4.1 Effect of Including the Lemma . . . 15
4.4.2 Effect of LoRA Target Modules . . . 15
4.5 Final Test Evaluation . . . 15
4.6 Error Analysis . . . 15
4.6.1 Per-class Performance and Confusions . . . 15
4.6.2 Analysis of Misclassified Examples . . . 18

5 Comparison of ICL and QLoRA **19**

5.1 Overall Results . . . 19
5.2 Limitations . . . 20

6 Conclusion **20**

1 Introduction

In this lab, we compare two approaches to French word-sense classification using large language models: in-context learning (ICL) and QLoRA fine-tuning. Our task is to assign one of 12 hypersense categories to a word sense based on its definition, with or without the lemma.

We use the same base model for both approaches. For ICL, we investigate how the number of demonstrations affects performance. For QLoRA, we compare different LoRA target modules. We also examine whether including the lemma improves classification in both approaches. Finally, we evaluate the selected configurations on two test sets and analyze their classification errors.

The complete implementation is available in our Google Colab notebook.

2 Dataset and Experimental Protocol

2.1 Dataset and classification task

We use a French word-sense dataset extracted from Wiktionary (Angleraud et al., 2025). For our experiments, we mainly use three fields: the lemma, its French definition, and its supersense. The dataset distinguishes 25 supersenses, which are mapped to 12 broader hypersense categories following Barque et al. (2020).

For example, *béruguette*, defined as *« Variété d'olive allongée à pulpe ferme »*, has the supersense `food`, which maps to the hypersense `inanimate`.

The dataset has an imbalanced class distribution, with some hypersenses appearing much more frequently than others. We therefore report Macro-F1 alongside Accuracy to better assess performance across all categories.

2.2 Data splits

The original dataset contains 17,159 word-sense examples divided into seven subsets. Following the lab instructions, we use four of them: `train`, `rand-dev`, `rand-test`, and `freq-test`. We exclude the `protect-frsemcor` subsets and `freq-dev`.

Examples with missing required information, such as a definition or hypersense label, are removed. Table 1 shows the number of examples before and after filtering, along with the purpose of each split.

Table 1: Dataset splits and their roles in the experiments.

Split	Original	Valid	Purpose
`train`	10,604	10,592	QLoRA training and ICL demonstrations
`rand-dev`	570	570	Configuration comparison and selection
`rand-test`	678	678	Final evaluation
`freq-test`	1,521	1,517	Final evaluation

The two test sets have different label distributions, allowing us to examine model performance under different evaluation conditions. Neither test set is used for configuration selection.

2.3 Model and experimental setup

We used the same base model for ICL and QLoRA. Our choice was based on the requirements of the classification task and the computational resources available.

We selected **Qwen3-4B-Instruct-2507** for three main reasons:

- **Multilingual capabilities:** Our task requires the model to understand French word definitions while following English instructions and predicting English hypersense labels. Qwen3 supports multilingual text processing, making it suitable for this task.
- **Instruction-following performance:** The model needs to follow a classification prompt and return one of 12 predefined labels. According to its official model card, Qwen3-4B-Instruct-2507 scores 69.0 on MultiIF, a benchmark evaluating multilingual instruction following. Although this benchmark does not directly evaluate French hypersense classification, it provides a useful indication of the model's instruction-following ability.
- **Model size and efficiency:** With 4 billion parameters, Qwen3 meets the lab's 8B limit while remaining practical for Google Colab. We chose the 4-bit quantized version from Unsloth to reduce GPU memory usage. This version is already quantized, so we did not need to perform quantization ourselves.

Experimental setup. Both methods were run on Google Colab with the following settings:

- **GPU & random seed:** NVIDIA T4; random seed 42.
- **Computation precision:** FP16 instead of the model's original BF16 configuration, as the T4 does not natively support BF16.

Due to limited GPU resources, ICL and QLoRA were run in separate Colab sessions. Their predictions were saved to Google Drive and later combined for evaluation and comparison.

2.4 Evaluation Metrics

We evaluate both methods primarily using accuracy and macro-F1. We also report the number of invalid predictions to check whether the model follows the required output format.

- **Accuracy:** The proportion of correctly classified examples. It measures overall performance but can be influenced by frequent categories.
- **Macro-F1:** The average F1-score across all 12 hypersenses, giving equal weight to each category. It is particularly useful for our imbalanced dataset.
- **Invalid predictions:** We also count outputs that do not match any of the 12 predefined hypersense labels. These predictions are treated as incorrect and remain included in the evaluation.

For error analysis, we also examine per-class **Precision, Recall, and F1-score**, along with class support, confusion matrices, and selected misclassified examples to identify common error patterns.

3 In-Context Learning

In-context learning (ICL) allows a language model to perform a task using instructions and examples in the prompt, without updating its parameters.

We investigate two questions: how the number of demonstrations affects classification performance, and whether adding the lemma helps the model classify French word senses. Before selecting one for final evaluation, we compare different configurations on `rand-dev`.

3.1 Prompting and inference

3.1.1 Prompt design

We adapted a classification prompt from Edwards and Camacho-Collados (2024) to our hypersense classification task. The instructions are written in English, while the word senses and their definitions remain in French. The prompt lists the 12 possible hypersense labels and asks the model to return only one category, without explanation.

We compare two input formats:

- **Definition only:** The model receives only the French definition.
- **Lemma + definition:** The model receives both the French lemma and its definition.

For few-shot prompting, labeled demonstrations are placed before the word sense to be classified. Figure 1 shows the general prompt template used across all configurations.

```
Your task is to assign a hypersense category to a French word sense.

Choose one category from the following list:
{hypersense_labels}

Provide only the category name. Do not include any explanation.

[FEW-SHOT ONLY]
Examples:

{demonstrations}
[END]

Now classify the following word sense.

[OPTIONAL] Lemma: {lemma}
Definition: {definition}
Category:
```

Figure 1: General prompt template for zero-shot and few-shot classification.

The demonstration section is omitted in zero-shot experiments and contains labeled examples in few-shot experiments. The lemma field is included only in the lemma + definition configuration, both in the demonstrations and in the final input.

The bracketed markers in Figure 1 indicate optional parts of the template and are not included in the actual prompts.

3.1.2 Prediction and label extraction

For each word sense, we construct a prompt using the selected input format and demonstrations, then generate a prediction with Qwen3-4B-Instruct-2507.

Since our task requires a single hypersense label, we use **greedy decoding** (`do_sample=False`) instead of sampling. Greedy decoding selects the most probable token at each step, reducing randomness and making comparisons across experimental configurations more consistent.

Since some hypersense labels contain multiple tokens, we set `max_new_tokens=24` as a practical limit without specifically optimizing this value. This allows enough room for multi-token hypersense labels while preventing unnecessarily long responses. The prompt still instructs the model to output only one label, allowing us to examine whether it produces a valid hypersense without strictly constraining its output to the predefined categories. These parameters are described in the Hugging Face Transformers documentation.

Although Qwen3-4B-Instruct-2507 is an instruction-tuned model, it may not always follow the required output format. We therefore apply a simple post-processing step: we take the first line of the generated response and use `strip()` to remove surrounding whitespace and punctuation. We then check whether the result exactly matches one of the 12 valid hypersense labels. Otherwise, we mark it as `INVALID` and count it as incorrect during evaluation.

3.2 Development experiments

3.2.1 Experimental design and baselines

We first compared different ICL configurations on `rand-dev`, varying two factors: the number of demonstrations (0-shot, 3-shot, and 6-shot) and the input format (definition only or lemma + definition).

For the few-shot experiments, we randomly sampled one example from each of the 12 hypersense categories in the `train` split. We then randomly selected six of these examples as our initial demonstrations. The 3-shot setting uses the first three examples, while the 6-shot setting uses all six. The 0-shot setting uses no demonstrations.

We used a fixed random seed of 42 and kept the same demonstrations for both input formats. This allows us to compare the effect of including the lemma without changing the examples.

We also included two simple baselines for comparison: a random baseline, which randomly assigns one of the 12 hypersense labels to each example, and a majority-class baseline, which always predicts the most frequent hypersense in the training set.

All configurations were evaluated on the same 570 examples from `rand-dev`, using accuracy, macro-F1, and the number of invalid predictions.

3.2.2 Initial results: 0, 3, and 6 shots

Table 2 presents the results of the six initial configurations and the two baselines.

Table 2: Initial ICL results on `rand-dev` (570 examples).

Configuration	Accuracy	Macro-F1	Invalid
Random baseline	0.0825	0.0571	0
Majority-class baseline	0.2386	0.0321	0
0-shot – definition only	0.4035	0.2736	6
0-shot – lemma + definition	0.3982	0.2835	14
3-shot – definition only	0.4825	0.2999	4
3-shot – lemma + definition	**0.5211**	0.3512	4
6-shot – definition only	0.4509	0.3009	3
6-shot – lemma + definition	0.4316	**0.3617**	4

All ICL configurations outperform both baselines in accuracy and macro-F1, including the zero-shot settings. This suggests that the model can already use the task instructions to make meaningful predictions without demonstrations.

Two observations stand out:

- **Number of demonstrations:** Few-shot prompting generally improves performance over zero-shot, but adding more examples does not always help. The 3-shot + lemma configuration achieves the highest accuracy (0.5211), while 6-shot + lemma obtains the highest macro-F1 (0.3617), despite its lower accuracy (0.4316).

- **Role of the lemma:** Including the lemma improves macro-F1 in all three shot settings. However, its effect on accuracy is mixed: it improves the 3-shot result but slightly reduces performance in the 0-shot and 6-shot settings.

Invalid predictions were relatively rare across all ICL configurations. The few-shot settings produced only 3–4 invalid predictions, compared with 6–14 in the zero-shot settings. This suggests that demonstrations may also help the model produce valid labels.

The results show no clear relationship between the number of demonstrations and classification performance. To explore this further, we added a 12-shot experiment.

3.2.3 Follow-up experiment: 12 shots

We kept the original six demonstrations and randomly selected one additional training example from each of the six hypersense categories not yet represented, using the same random seed of 42. This gave us 12 demonstrations covering all 12 hypersense categories, with one example per category. We tested both input formats on `rand-dev`, with the results shown in Table 3.

Table 3: Follow-up 12-shot results on `rand-dev`.

Configuration	Accuracy	Macro-F1	Invalid
12-shot – definition only	0.4351	0.3438	2
12-shot – lemma + definition	0.4491	**0.3637**	7

Increasing to 12 demonstrations did not improve upon the highest accuracy achieved with 3 shots. The 12-shot + lemma configuration achieved an accuracy of 0.4491, lower than the 3-shot + lemma result (0.5211).

However, 12-shot + lemma achieved the highest macro-F1 (0.3637) among all configurations, although the improvement over 6-shot + lemma (0.3617) was very small. This suggests that using more demonstrations may affect accuracy and macro-F1 differently, but the results do not show a clear overall benefit.

Invalid predictions remained relatively rare (2 for definition only and 7 for lemma + definition), but increasing the number of demonstrations did not consistently reduce them.

An important limitation is that increasing the number of demonstrations also increases the number of hypersense categories represented in the prompt. Therefore, we cannot determine whether the observed changes come from the number of examples, the broader category coverage, or both.

3.2.4 Optional Experiment: Generation vs. Label Log-Probability

In greedy label generation, the model predicts tokens one by one from its entire vocabulary. Although the prompt asks it to output only a hypersense label, it may generate additional text or an answer outside the 12 categories. We therefore explore another inference method: instead of generating an answer, we directly compare the probabilities of the candidate labels and select the most likely one.

The lab instructions suggest comparing label log-probabilities **at the position immediately after the prompt**, which requires single-token labels. However, some of our original hypersense labels contain multiple tokens. For example, `dynamic_situation` is split into four tokens by the Qwen tokenizer. Its probability cannot be obtained from a single next-token prediction; calculating it would require scoring the entire token sequence. To follow the method described in the instructions, we replace the 12 hypersense names with **single-token codes**

(A–L). The prompt includes the mapping between each code and its original category, and the demonstrations use these codes as their answers.

In greedy label generation, the model generates an answer normally, with `max_new_tokens=8` instead of 24, since the A–L codes are single tokens. We then extract the predicted code and convert it back to its original hypersense label.

We compare two inference methods using the same **3-shot + lemma configuration** on `rand-dev`. Both use the same model, prompt, category mapping, and three demonstrations. In **greedy label generation**, the model generates an answer normally, and we extract the predicted code and convert it back to its original hypersense label. In **label log-probability scoring**, we obtain the logits for the next token after the prompt, apply log-softmax, and compare the scores of the 12 candidate codes. The code with the highest score is selected as the prediction. Since the choice is restricted to A–L, this method always returns a valid category. The results of this comparison are shown in Table 4.

Table 4: Comparison of generation and logprob methods on the rand-dev.

Method	Accuracy	Macro-F1	Invalid
Generation	0.4386	0.2763	0
Logprob	0.4386	0.2763	0

Both methods achieved an accuracy of 0.4386 and a macro-F1 of 0.2763 on `rand-dev`, with no invalid predictions. For log-probability scoring, this was expected because the prediction was restricted to the 12 valid A–L codes. Interestingly, greedy generation also produced no invalid predictions, even though its output was not restricted to these codes.

More importantly, both methods produced exactly the same predictions for all 570 examples. This is consistent with the behavior of greedy decoding: when the model selects a valid A–L token as its most probable next token, restricting the choice to these 12 candidates does not change the prediction. In our experiment, log-probability scoring therefore provided no additional benefit over greedy generation.

Both methods also performed worse than the original 3-shot configuration using hypersense names (accuracy: 0.5211; macro-F1: 0.3512). One possible explanation is that the original category names provide useful semantic information, while the A–L codes require the model to rely on the mapping given in the prompt.

3.3 Final Configuration Selection

We selected 3-shot + lemma + definition with greedy generation for the final evaluation. It achieved the highest accuracy (0.5211) on `rand-dev`, with a macro-F1 of 0.3512, close to the best result of 0.3637 obtained with 12 shots. Adding more demonstrations did not consistently improve performance.

The additional A–L experiments performed worse (accuracy: 0.4386; macro-F1: 0.2763), and log-probability scoring produced the same predictions as greedy generation. We therefore kept the original hypersense labels and applied the selected configuration unchanged to `rand-test` and `freq-test`.

3.4 Final Evaluation

3.4.1 Results on `rand-test` and `freq-test`

We evaluated the selected 3-shot + Lemma + Definition configuration on the two test sets, without further changes to the prompt or demonstrations. The results are shown in Table 5.

Table 5: Final ICL results on rand-test and freq-test.

Test Set	Examples	Accuracy	Macro-F1	Invalid
rand-test	678	0.5413	0.3686	3
freq-test	1,517	0.5049	0.3876	10

The model achieved higher accuracy on `rand-test` (0.5413) than on `freq-test` (0.5049). However, its macro-F1 was slightly higher on `freq-test` (0.3876 versus 0.3686).

This difference suggests that overall accuracy does not fully reflect performance across the 12 hypersense categories. We examine the per-class results and confusion matrices below to better understand these differences.

3.4.2 Error Analysis

We looked at the per-class F1 scores and confusion matrices to examine the mistakes made by our selected ICL model. The results show clear differences across hypersenses, as well as some interesting patterns in how the model assigns labels.

Per-class Performance and Differences Between Test Sets

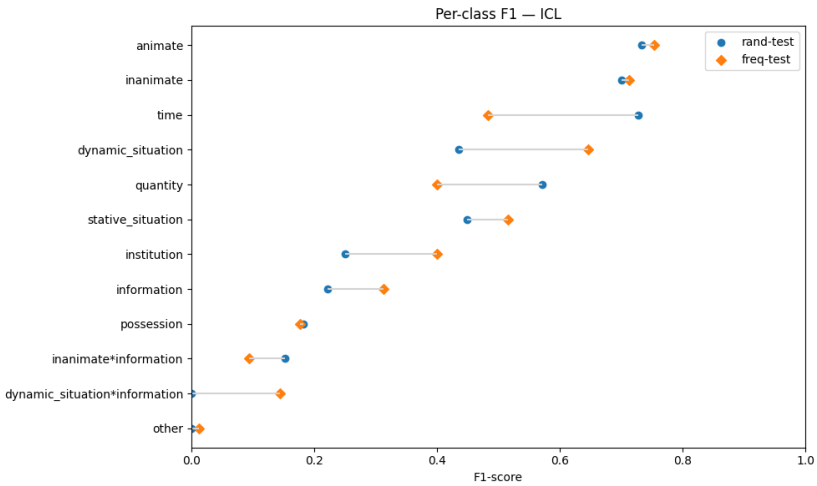


Figure 2: Per-class F1 scores of the selected ICL model across all 12 hypersenses on rand-test and freq-test.

Table 6: Per-class F1 scores and support on both test sets.

Hypersense	rand-test F1	rand-test Support	freq-test F1	freq-test Support
`animate`	0.734	218	0.754	127
`dynamic_situation`	0.435	77	0.646	353
`dynamic_situation*information`	0.000	3	0.145	46
`inanimate`	0.700	214	0.713	317
`inanimate*information`	0.153	17	0.094	37
`information`	0.222	45	0.313	163
`institution`	0.250	7	0.400	28
`other`	0.000	26	0.012	163
`possession`	0.182	3	0.177	22
`quantity`	0.571	14	0.400	46
`stative_situation`	0.449	47	0.515	196
`time`	0.727	7	0.483	19
Macro-F1	**0.369**	–	**0.388**	–

Figure 2 compares the per-class F1 scores of the selected ICL model on both test sets. Table 6 provides the exact F1 scores and support (the number of true instances in each category).

(1) Performance varies across hypersenses.

The model performed relatively well on `animate` and `inanimate`, with F1 scores above 0.70 on both test sets. However, it struggled with `information`, `inanimate*information`, and especially `other`. This uneven performance partly explains why macro-F1 was much lower than accuracy. Some results should also be interpreted carefully. For example, `time` and `quantity` achieved F1 scores of 0.727 and 0.571 on rand-test, but these were based on only 7 and 14 instances, respectively.

The model particularly struggled with `other`. The model also seems to prefer assigning a specific hypersense rather than choosing the more general `other` label, even when its prediction is incorrect. It failed to correctly classify any of its 26 instances on rand-test and identified only 1 out of 163 on freq-test. In fact, it predicted `other` only four times on freq-test, often assigning these instances to more specific categories such as `dynamic_situation` or `inanimate`. One possible explanation is that `other` covers a wide range of meanings without clear shared semantic features, making it harder for the model to recognize.

(2) Differences between rand-test and freq-test.

The model achieved higher accuracy on rand-test (0.541) than on freq-test (0.505), but its macro-F1 was slightly lower on rand-test (0.369) than on freq-test (0.388). The differences also appeared in individual categories. For example, `dynamic_situation` had an F1 score of 0.435 on rand-test and 0.646 on freq-test, while `time` showed the opposite pattern (0.727 on rand-test and 0.483 on freq-test).

Two possible factors are label distribution and word frequency.

First, the label distributions differ between the two datasets. For example, `animate`, which the model handles relatively well, accounts for about 32% of rand-test but only 8% of freq-test. In contrast, `other`, one of the most difficult categories, is more common in freq-test. This may partly explain the lower accuracy on freq-test.

Word frequency may also affect the results. Angleraud et al., 2025 reported an average of 1.26 senses per lemma in the random subset, compared with 3.20 in the frequency-based subset. Frequent words may be more familiar to the model because of pretraining, but having more meanings could also make them harder to classify.

However, we can not conclude from these results alone that frequent words are easier or harder to classify. A more detailed analysis controlling for label distribution and polysemy would be needed to better understand the effect of word frequency.

Confusion Matrix Analysis

We also examined the confusion matrices to see which categories were most frequently mixed up. Figures 3 and 4 show the row-normalized confusion matrices as heatmaps. Rows represent true labels, columns represent predicted labels, and values indicate the percentage of instances in each true class. Since the full matrices contain many values, we only include the heatmaps here. The detailed confusion matrices with raw counts are available in our Colab notebook.

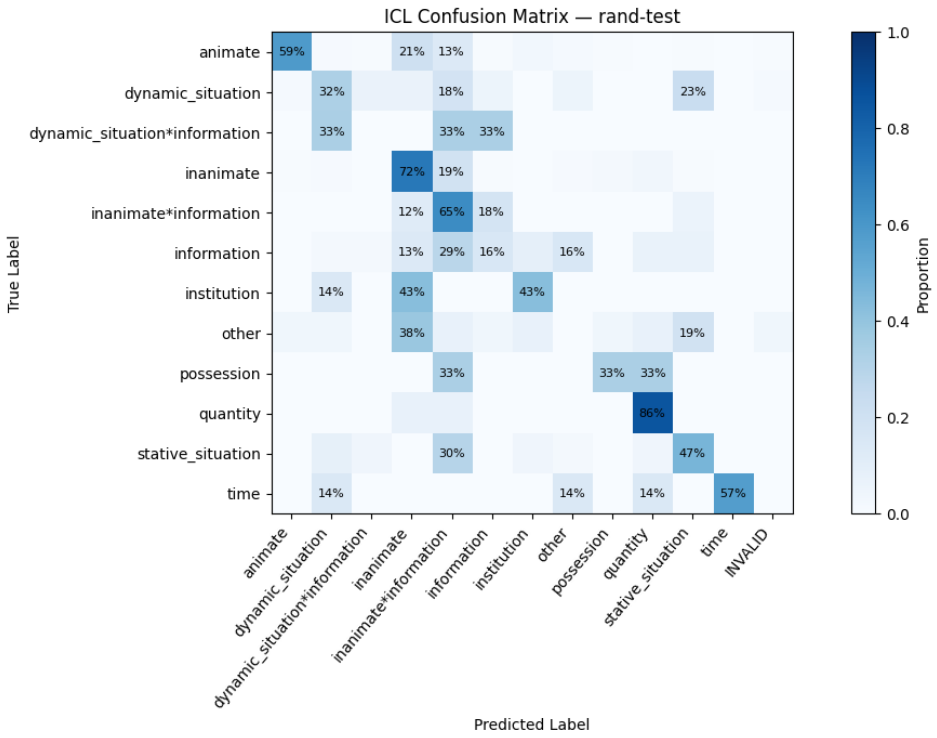


Figure 3: Confusion matrix heatmap of the selected ICL model on `rand-test`.

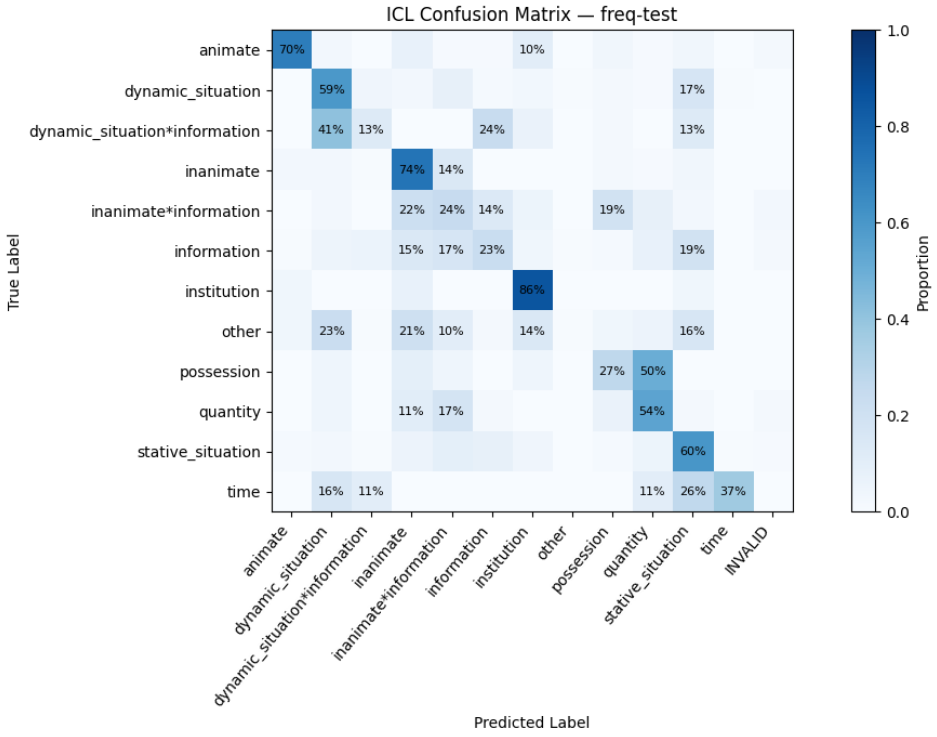


Figure 4: Confusion matrix heatmap of the selected ICL model on `freq-test`.

(3) Confusion between semantic categories

Some errors occurred between related semantic categories. For example, on rand-test, 20.6% (45/218) of `animate` instances were incorrectly classified as `inanimate`, compared with only 0.5% (1/214) in the opposite direction. This asymmetry was less pronounced on freq-test, with corresponding error rates of 7.1% (9/127) and 2.5% (8/317).

We also noticed confusion between `dynamic_situation` and `stative_situation`. The model incorrectly predicted `stative_situation` for 23.4% (18/77) of `dynamic_situation` instances on rand-test and 16.7% (59/353) on freq-test. This may reflect difficulties in distinguishing events or processes from states.

(4) Difficulties with combined hypersenses

The model also struggled with combined hypersenses, but in different ways.

For `inanimate*information`, the model seemed to overuse this label. On rand-test, it predicted this category 127 times, although only 17 instances actually belonged to it. A similar pattern appeared on freq-test (154 predictions for 37 true instances). On rand-test, 19.2% (41/214) of `inanimate` and 28.9% (13/45) of `information` instances were incorrectly assigned to this combined category. The same problem appeared on freq-test, with 14.2% (45/317) and 16.6% (27/163), respectively.

In contrast, `dynamic_situation*information` was often classified as one of its component categories. On freq-test, only 13.0% (6/46) were correctly classified, while 41.3% (19/46) were predicted as `dynamic_situation` and 23.9% (11/46) as `information`. This suggests that the model has difficulty deciding when to use a single or combined hypersense. Sometimes it uses a combined label too broadly, while in other cases it chooses only one of its components.

Invalid Predictions

Invalid predictions were relatively rare, accounting for only 0.4% (3/678) on rand-test and 0.7% (10/1517) on freq-test. Table 7 shows some examples of invalid outputs.

Table 7: Examples of invalid predictions from the selected ICL model.

Test set	Lemma	True label	Raw output
rand-test	écho	`dynamic_situation`	`sound`
rand-test	lessive	`other`	`possessioin`
freq-test	chimie	`information`	`science`
freq-test	remède	`inanimate`	`possessing`
freq-test	relation	`stative_situation`	`relation: inanimate*information`
freq-test	forme	`information`	`category: inanimate*information`
freq-test	création	`inanimate*information`	`possessio`

As shown in Table 7, the model produced several types of invalid outputs. In some cases, it generated words outside the predefined labels, such as `sound` for *écho* and `science` for *chimie*. It also produced misspelled or incomplete labels, such as `possessioin` and `possessio`, instead of `possession`. In other cases, the output contained a valid hypersense but included extra text, such as `category: inanimate*information`. These outputs were still marked as invalid because they did not exactly match the required label format.

Although the prompt explicitly asked for only one hypersense label, the model did not always follow this instruction. However, more than 99% of its predictions were valid on both test sets. Classification errors mainly came from selecting an incorrect hypersense.

Misclassified Examples

Understanding why a model makes a particular classification error is not easy, since the final prediction alone does not tell us which features the model relied on.

To explore these errors, we selected three common confusion patterns from the confusion matrix: `animate` to `inanimate`, `dynamic_situation` to `stative_situation`, and `inanimate` to `inanimate*information`.

For each pattern, we randomly sampled five misclassified examples and manually selected one for closer examination. The three selected examples are shown in Table 8. Our explanations are based on the definitions and possible semantic similarities between categories. They are hypotheses but not the confirmed reasons for the model's predictions.

Table 8: Selected misclassified ICL examples on rand-test.

Lemma	**Definition**	**Gold label**	**Prediction**
hack	Cheval monté pour surveiller les performances et le comportement des autres chevaux.	`animate`	`inanimate`
tachyarythmie	Combinaison d'une accélération et d'une irrégularité du rythme cardiaque.	`dynamic_situation`	`stative_situation`
aimantin	Objet décoratif aimanté.	`inanimate`	`inanimate*information`

For *hack*, the definition clearly refers to a horse, which should belong to `animate`. The model predicts `inanimate`. One possible explanation is that the word *hack* has other meanings, such as in computing, which may have influenced the prediction despite the explicit reference to an animal in the definition. However, we cannot confirm this from the output alone.

For *tachyarythmie*, the definition describes a combination of an accelerated and irregular heart rhythm. Although it involves changes in heart rhythm, it can also be understood as an ongoing medical condition. The model may focus more on this state-like meaning, which could explain why it predicts `stative_situation` instead of `dynamic_situation`. This example shows how the distinction between a process and a state may not always be clear.

For *aimantin*, the definition describes a magnetic decorative object, without any obvious information-related meaning. The model predicts `inanimate*information` instead of `inanimate`. It may associate decorative artifacts with some informational or representational function, although nothing in this definition clearly supports that interpretation.

4 QLoRA Fine-tuning

4.1 Method and Training Setup

We use QLoRA (Dettmers et al., 2023) to fine-tune the same instruction-tuned causal model used for ICL, `unsloth/Qwen3-4B-Instruct-2507-bnb-4bit`. The 4-bit base model remains frozen, and only the LoRA adapter parameters are updated during training. We compare adaptation of the attention projections alone with adaptation of both attention and feed-forward network (FFN) projections.

For an original weight matrix W_0, LoRA represents its update using two smaller trainable matrices as $W = W_0 + \frac{\alpha}{r}BA$, where W_0 remains frozen and only A and B are updated. We use $r = 16$ and $\alpha = 16$, giving a scaling factor of $\alpha/r = 1$.

Our adapter setup uses the same preparation and wrapping steps as the Hugging Face PEFT quantization guide: we prepare the quantized model with `prepare_model_for_kbit_training`, configure LoRA with `LoraConfig`, and attach the adapters with `get_peft_model`.

We formulate classification as text generation. The prompt contains the task instruction, the lemma when included, the definition, and the list of possible hypersense labels. The target is the correct hypersense label followed by an end-of-sequence token. The prompt tokens are excluded from the loss calculation, which covers only the answer and its end-of-sequence token. We use the same prompt format during training and inference to keep the input structure consistent. At inference time, the correct label is omitted and generated by the model. We use `Trainer` from Hugging Face Transformers to train the LoRA adapters and evaluate validation

loss on `rand-dev`.

The adapters are independently initialized from the same base model. Each experiment uses the full valid training set of 10,592 examples. Table 9 lists the training hyperparameters shared across the three experimental configurations.

Table 9: Common training settings for the QLoRA experiments.

Parameter	**Value**
LoRA rank	16
LoRA alpha	16
LoRA dropout	0.05
Learning rate	2×10^{-4}
Learning-rate schedule	Cosine
Warmup steps	40
Optimizer	Paged AdamW, 8-bit
Training epochs	1
Batch size per device	1
Gradient accumulation steps	8
Effective batch size	8
Maximum sequence length	256 tokens
Development set	`rand-dev` (570 examples)
Evaluation and saving	Every 250 steps and at the final step
Random seed	42

These settings were chosen to train a 4B model on a Colab T4 GPU. The LoRA rank limits the number of trainable parameters, while a dropout rate of 0.05 provides light regularization. A batch size of one fits within GPU memory, and eight gradient accumulation steps give an effective batch size of eight. We use fixed training hyperparameters without a hyperparameter search.

Each adapter is evaluated and saved every 250 optimization steps and at the final step. The checkpoint with the lowest validation loss on `rand-dev` is retained. We compare the configurations using development Macro-F1 and reserve the test sets for the final evaluation.

4.2 Experimental Design

We conduct three experiments, summarized in Table 10. All experiments generate the hypersense label as text during prediction.

Table 10: QLoRA experiments.

Experiment	**Input**	**LoRA target modules**
A	Lemma + definition	Attention + FFN
B	Definition only	Attention + FFN
C	Lemma + definition	Attention only

4.2.1 RQ1: Does Including the Lemma Help?

Experiments A and B examine whether including the lemma improves classification. Experiment A receives both the lemma and definition, while Experiment B receives only the definition. Both use independently initialized LoRA adapters with the same base model and training settings. The task instruction, definition, list of possible labels, and target output remain unchanged.

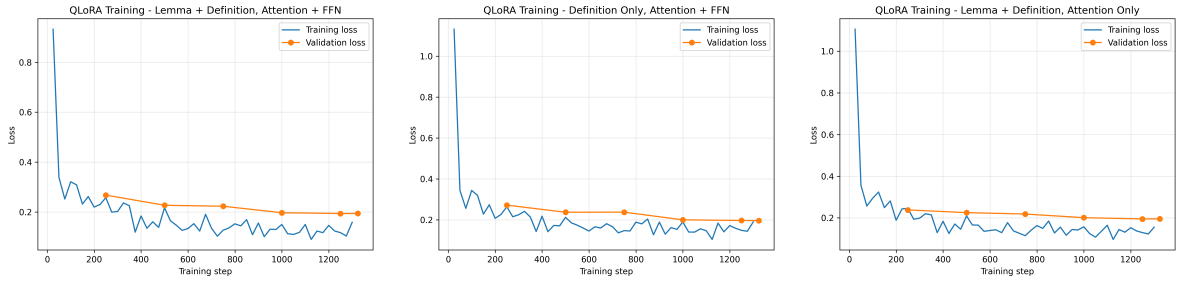
Both experiments apply LoRA to the attention projections (`q_proj`, `k_proj`, `v_proj`, and `o_proj`) and the FFN projections (`gate_proj`, `up_proj`, and `down_proj`).

4.2.2 RQ2: Does Extending LoRA to the FFN Help?

Experiments A and C examine whether extending LoRA adaptation to the FFN improves classification. Both receive the lemma and definition and use the same training settings. Experiment A applies LoRA to both the attention and FFN projections, while Experiment C applies LoRA only to `q_proj`, `k_proj`, `v_proj`, and `o_proj`.

This comparison changes the adaptation scope and the number of trainable parameters. We therefore report parameter counts alongside development-set classification performance. Experiment C serves as a supplementary comparison of LoRA target modules.

4.3 Training Results



(a) Exp. A: lemma and definition, Attention + FFN.

(b) Exp. B: definition only, Attention + FFN.

(c) Exp. C: lemma and definition, Attention only.

Figure 5: Training and validation loss for the three QLoRA experiments.

After preprocessing, the training set contains 10,592 examples. With an effective batch size of eight, one epoch corresponds to 1,324 optimization steps. Figure 5 shows the training and validation loss for the three experiments. Training loss is logged every 25 steps, while validation loss is evaluated every 250 steps and at the final step.

Across the three experiments, the training loss drops quickly at first and then fluctuates at a relatively low level. The validation loss decreases more gradually, with no sustained increase during training. The three experiments show similar trends, with only small differences in their minimum validation losses.

Experiment A: Lemma and definition, Attention + FFN. The lowest validation loss is 0.1937 at step 1,250, and the corresponding adapter is retained.

Experiment B: Definition only, Attention + FFN. The lowest validation loss is 0.1968 at the final training step, 1,324. The adapter saved at this step is retained.

Experiment C: Lemma and definition, Attention only. The lowest validation loss is 0.1949 at step 1,250, and the corresponding adapter is retained.

4.4 Results on `rand-dev`

Table 11: QLoRA results across the three experiments on `rand-dev`.

Experiment	Trainable parameters	Accuracy	Macro-F1	Invalid
A	33,030,144	0.8561	**0.6462**	0
B	33,030,144	0.8509	0.6308	0
C	11,796,480	**0.8579**	0.6312	0

4.4.1 Effect of Including the Lemma

As shown in Table 11, on `rand-dev`, Experiment A obtains an Accuracy of 0.8561 and a Macro-F1 of 0.6462. Experiment B obtains an Accuracy of 0.8509 and a Macro-F1 of 0.6308. Both produce no invalid predictions.

Including the lemma improves Accuracy from 0.8509 to 0.8561 and Macro-F1 from 0.6308 to 0.6462. The improvement is modest but observed in both metrics. In particular, the higher Macro-F1 indicates a small improvement in average performance across the 12 hypersense classes, suggesting that the lemma may provide useful lexical information in addition to the definition under the Attention + FFN configuration.

4.4.2 Effect of LoRA Target Modules

As shown in Table 11, on `rand-dev`, Experiment A (Attention + FFN) obtains an Accuracy of 0.8561 and a Macro-F1 of 0.6462, while Experiment C (Attention only) obtains an Accuracy of 0.8579 and a Macro-F1 of 0.6312. Both produce no invalid predictions.

Experiment A has 33,030,144 trainable parameters, compared with 11,796,480 for Experiment C, which uses approximately 64.3% fewer parameters. Although Experiment C achieves slightly higher Accuracy, Experiment A obtains higher Macro-F1. These results suggest that extending LoRA to the FFN can improve average class-level performance under the evaluated settings, at the cost of more trainable parameters.

4.5 Final Test Evaluation

We use Macro-F1 as the main criterion for model selection because the hypersense classes are imbalanced. Experiment A achieves the highest development Macro-F1 among the three configurations, although Experiment C obtains slightly higher Accuracy with fewer trainable parameters. We therefore select **Experiment A** for final test evaluation. Experiment A uses both the lemma and definition, with LoRA applied to the attention and FFN projections.

Table 12: Final test results for Experiment A (lemma + definition, Attention + FFN).

Split	Examples	Accuracy	Macro-F1	Invalid
`rand-test`	678	0.8658	0.6985	0
`freq-test`	1,517	0.7462	0.6773	0

Only the adapter from Experiment A is evaluated on `rand-test` and `freq-test`. As shown in Table 12, the selected adapter obtains an Accuracy of 0.8658 and a Macro-F1 of 0.6985 on `rand-test`. On `freq-test`, it obtains an Accuracy of 0.7462 and a Macro-F1 of 0.6773. It produces no invalid predictions on either test set. Performance is lower on `freq-test`, especially in terms of Accuracy. This difference may partly reflect the different class distributions of the two test sets, as discussed in Section 4.6.

4.6 Error Analysis

4.6.1 Per-class Performance and Confusions

Per-class Performance

Table 13: Selected per-class results for Experiment A on `rand-test` and `freq-test`.

	rand-test				freq-test			
Hypersense	**Precision**	**Recall**	**F1**	**Support**	**Precision**	**Recall**	**F1**	**Support**
`animate`	0.9775	0.9954	0.9864	218	0.9322	0.8661	0.8980	127
`inanimate`	0.8940	0.9065	0.9002	214	0.7832	0.9117	0.8426	317
`institution`	0.5000	0.7143	0.5882	7	0.3200	0.8571	0.4660	28
`dynamic_situation*information`	0.5000	0.3333	0.4000	3	0.7500	0.3913	0.5143	46
`other`	1.0000	0.0769	0.1429	26	0.6341	0.1595	0.2549	163

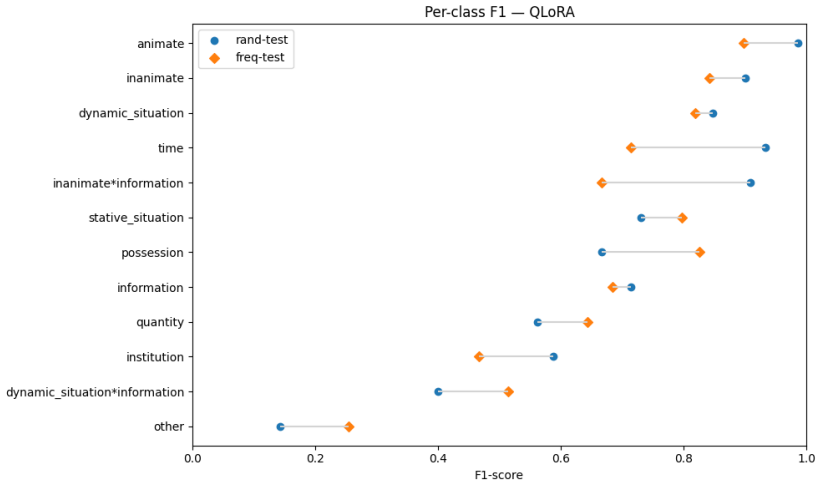


Figure 6: Per-class F1-scores of Experiment A on `rand-test` and `freq-test`.

Table 13 and Figure 6 show differences in performance across the 12 hypersense classes.

The model achieves high F1-scores on `animate` and `inanimate` in both test sets, while `other` has the lowest F1-scores. The good performance on `inanimate` may be partly related to its large number of examples in the training set, which gave the model more opportunities to learn this category. However, we would need further analysis to see whether this is the case.

For `other`, the main difficulty is low Recall. On `rand-test`, its Precision is 1.0000, but Recall is only 0.0769, meaning that only 2 of 26 examples are correctly identified. On `freq-test`, Recall remains low at 0.1595, with 26 of 163 examples correctly classified.

We should be careful when interpreting results for categories with very few examples. For example, `dynamic_situation*information` has only three examples in `rand-test`, so a single prediction can substantially change its score.

Differences Between Test Sets

The class distributions differ between the two test sets. Together, `animate` and `inanimate` account for 63.7% of `rand-test`, compared with 29.3% of `freq-test`. Since the model performs relatively well on these categories, this difference may partly explain the higher overall Accuracy on `rand-test`.

The per-class F1-scores do not change uniformly across the two test sets. For example, the F1-score of `animate` decreases from 0.9864 on `rand-test` to 0.8980 on `freq-test`, while that of `other` increases from 0.1429 to 0.2549. However, we should be careful when comparing these results because the two test sets have different numbers of examples in each category.

Confusion Patterns

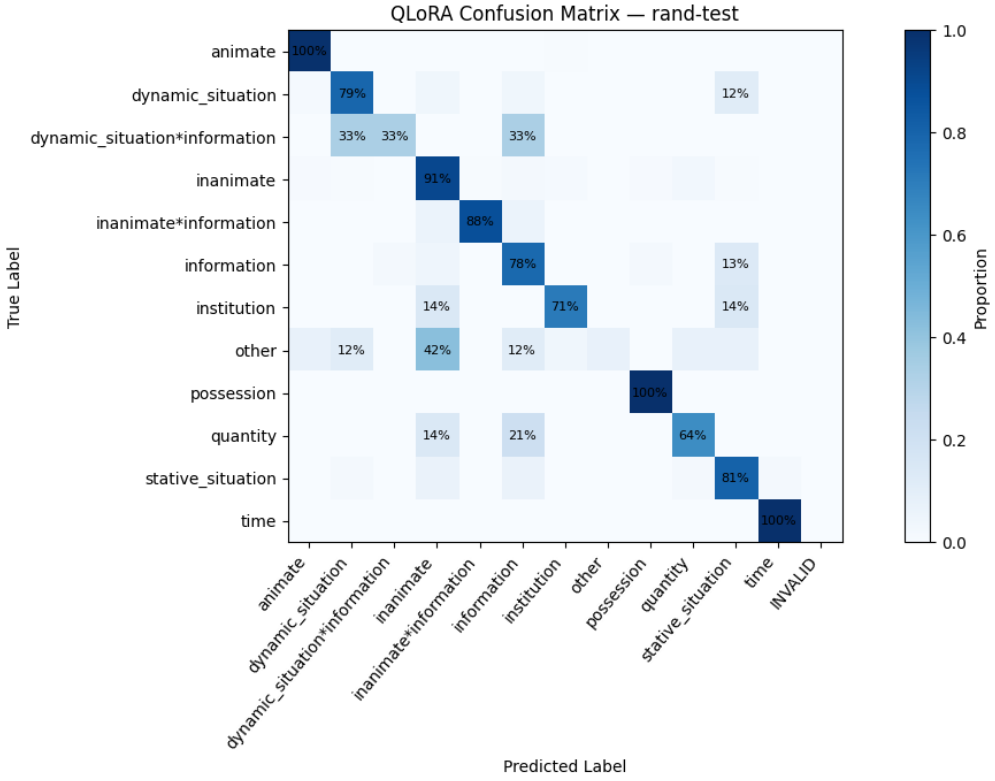


Figure 7: Row-normalized confusion matrix for Experiment A on `rand-test`.

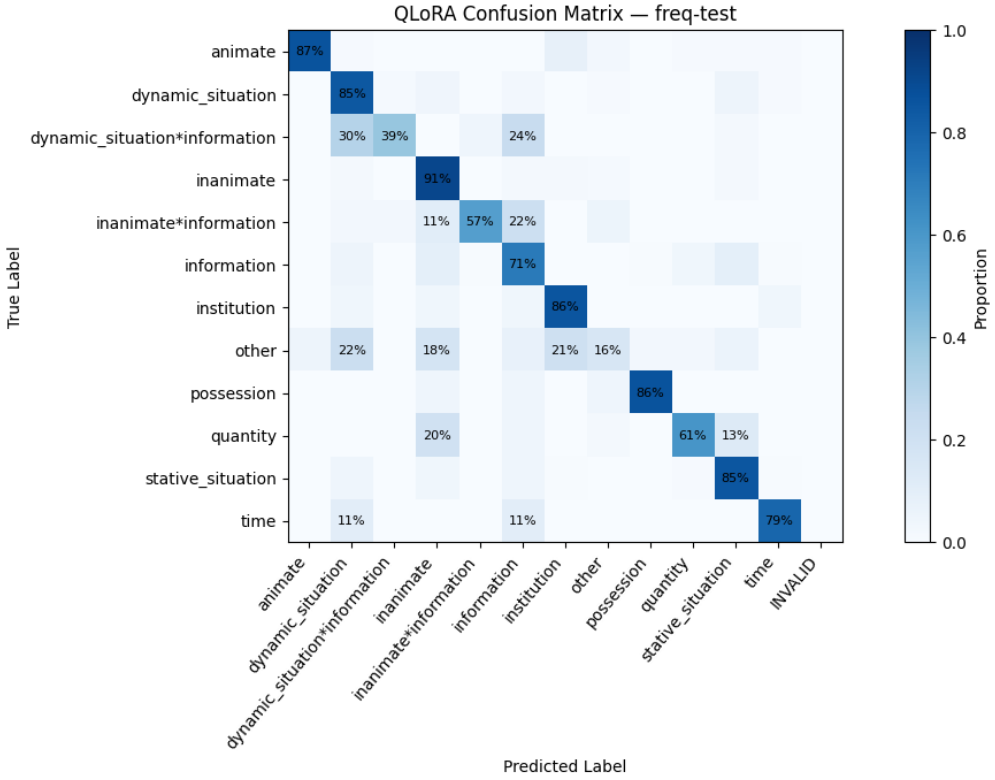


Figure 8: Row-normalized confusion matrix for Experiment A on `freq-test`.

Figures 7 and 8 show the row-normalized confusion matrices for the two test sets.

First, the model frequently misclassifies `other` as other hypersenses. Only 8% (2/26) of `other` examples are correctly identified on `rand-test`, compared with 16% (26/163) on `freq-test`. On `rand-test`, 42% are incorrectly classified as `inanimate`. On `freq-test`, `other` is frequently predicted as `dynamic_situation` (22%), `institution` (21%), or `inanimate` (18%). The heterogeneous nature of `other` may contribute to these difficulties, although individual examples need to be examined to assess this explanation.

Second, some errors involve combined hypersenses. For `dynamic_situation*information` on `freq-test`, only 39% (18/46) of examples are correctly classified, while 30% (14/46) are predicted as `dynamic_situation` and 24% (11/46) as `information`. A similar confusion appears on `rand-test`, where two of the three examples are assigned to these individual components. However, the very small support on `rand-test` makes this result difficult to interpret. These results suggest that the model may have difficulty distinguishing this combined hypersense from its individual components.

Finally, `inanimate` is a frequent destination of incorrect predictions from several hypersense categories. These include `other`, `information`, and `dynamic_situation`, particularly on `freq-test`. Although the model achieves high Recall for `inanimate`, it also incorrectly assigns this label to examples belonging to other categories.

4.6.2 Analysis of Misclassified Examples

To better understand the confusion patterns observed above, we selected three misclassified examples from `freq-test`, along with one correctly classified example for comparison (see Table 14).

Table 14: Selected predictions from Experiment A on `freq-test`.

Lemma	Definition	Gold label	Prediction
rouleau	Objet de forme cylindrique oblongue.	`other`	`inanimate` ✗
conseil	Avis que l'on donne à quelqu'un sur ce qu'il doit faire ou ne pas faire.	`dynamic_situation *information`	`information` ✗
champ	Espace éditable dans un formulaire.	`information`	`inanimate` ✗
champ	Colonne de table de base de données, avec un type.	`information`	`information` ✓

These examples illustrate some of the confusion patterns observed above. For *rouleau*, the definition describes a physical object, which may explain why the model predicts `inanimate` instead of `other`. For *conseil*, the model selects `information`, possibly focusing on the content of the advice without identifying the associated action represented by the combined label.

The two examples of *champ* provide an interesting comparison. **Although they share the same lemma and gold label, their definitions differ, and the model produces different predictions.** It incorrectly predicts `inanimate` for the definition describing an editable space in a form, but correctly identifies `information` when the definition refers to a database column. The more explicit database context may help the model identify the information-related meaning. This suggests that the model is sensitive to semantic cues in different definitions, but does not always identify the correct hypersense.

However, these examples alone cannot determine which cues the model actually relies on.

5 Comparison of ICL and QLoRA

5.1 Overall Results

Table 15 compares the final results of ICL and QLoRA on both test sets. Both methods use the same base model. For ICL, we use the selected 3-shot + Lemma + Definition configuration, providing three labeled examples in the prompt without updating the model parameters. For QLoRA, we use the selected Lemma + Definition configuration, with LoRA applied to both the attention and FFN projections during fine-tuning.

Table 15: Final test results of ICL and QLoRA.

Method	Test set	Accuracy	Macro-F1	Invalid
ICL	`rand-test`	0.5413	0.3686	3
ICL	`freq-test`	0.5049	0.3876	10
QLoRA	`rand-test`	**0.8658**	**0.6985**	0
QLoRA	`freq-test`	**0.7462**	**0.6773**	0

QLoRA achieves substantially higher accuracy and macro-F1 than ICL on both test sets. It also produces no invalid predictions, compared with 3 on rand-test and 10 on freq-test for ICL. Both methods achieve higher accuracy on rand-test, but their macro-F1 scores show different patterns: ICL achieves a slightly higher score on freq-test than on rand-test (0.3876 vs. 0.3686), whereas QLoRA achieves a slightly higher score on rand-test than on freq-test (0.6985 vs. 0.6773).

To compare the two methods more directly, we also examined whether ICL and QLoRA correctly classified the same test examples. Figure 9 shows the paired prediction outcomes.

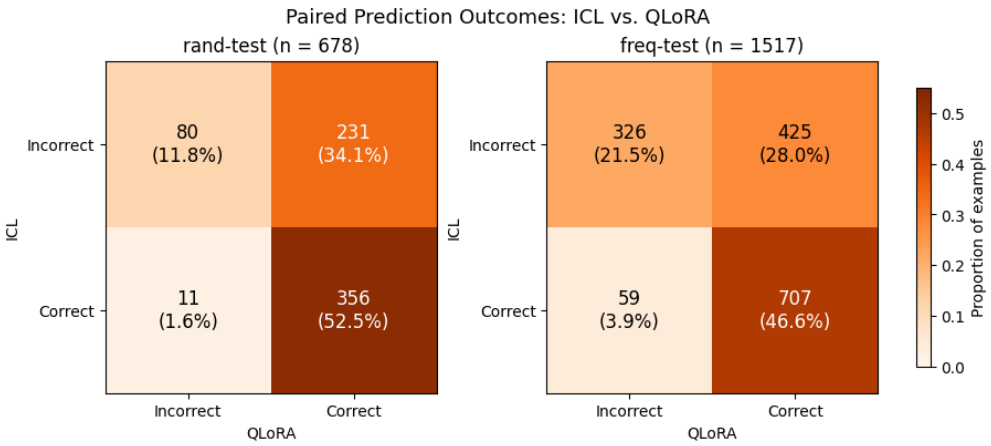


Figure 9: Paired prediction outcomes of ICL and QLoRA on `rand-test` and `freq-test`. Each cell shows the number and percentage of examples.

On `rand-test`, QLoRA correctly classified 231 examples that ICL predicted incorrectly, whereas ICL correctly classified only 11 examples that QLoRA missed. A similar pattern appears on `freq-test`, with 425 examples correctly classified only by QLoRA, compared with 59 only by ICL. These results further demonstrate the higher accuracy of QLoRA on both test sets.

However, both methods misclassified 80 of the same examples on `rand-test` and 326 on `freq-test`, showing that their errors also overlap on some examples.

5.2 Limitations

Our experiments have some limitations, mainly due to limited computational resources and time. For ICL, we used a fixed set of demonstrations and did not test how different example selections might affect performance. For QLoRA, we trained each configuration only once, so we could not assess how stable the results would be across different random seeds.

In future experiments, we could try different demonstrations for ICL and repeat QLoRA training with different random seeds to see whether the results remain consistent.

6 Conclusion

In this experiment, QLoRA achieved higher Accuracy and Macro-F1 on both test sets. Our results show that including the lemma improved Macro-F1 for both methods, suggesting that it provides extra information beyond the definition. However, both methods still had difficulties with certain hypersense categories, particularly `other` and some combined hypersenses.

Both methods achieved lower Accuracy on `freq-test` than on `rand-test`. One possible explanation is the difference in class distributions. For example, `animate`, a category both methods handled relatively well, accounts for about 32% of `rand-test` but only 8% of `freq-test`, while the more difficult `other` category is more common in `freq-test`. Another possible explanation involves word frequency and polysemy. Intuitively, frequent words might be easier for the model to classify because they are likely to appear more often during pretraining. However, we found that frequent words also tend to have more senses. Angleraud et al., 2025 report an average of 1.26 senses per lemma in the random subset, compared with 3.20 in the frequency-based subset. Since `freq-test` focuses on frequent words, this greater polysemy may make classification more difficult and partly explain its lower Accuracy. However, further analysis would be needed to assess how much class distribution and polysemy each contribute to the differences in classification performance.

For ICL, few-shot prompting generally worked better than zero-shot prompting, but more demonstrations did not always lead to better results. The 3-shot + Lemma + Definition configuration achieved the highest accuracy on rand-dev and was selected for final evaluation. The 12-shot configuration achieved the highest macro-F1, but the improvement over 3-shot was very small. We also compared greedy generation and log-probability scoring using single-token A–L labels. Both methods produced exactly the same predictions, but performed worse than the original configuration using hypersense names. This suggests that the way labels are represented may also influence classification performance.

For QLoRA, adapting both attention and FFN projections gave a higher Macro-F1 than attention alone. We therefore selected this configuration with both lemma and definition for final evaluation. However, the attention-only configuration achieved slightly higher Accuracy and only slightly lower Macro-F1 on the development set, while using about 64% fewer trainable parameters. This shows that adapting attention projections alone can still give good results with far fewer parameters. We found that `inanimate` was by far the most frequent category in the training set. This may partly explain why the model performed well on this category, but further work would be needed to confirm this explanation. The error analysis also showed that different definitions of the same word received different predictions, even though they had the same gold hypersense label. This suggests that the model may be sensitive to semantic cues in definitions, although these examples alone cannot tell us which cues it relies on.

This experiment also made us wonder whether different attention projection modules might contribute differently to fine-tuning. We tested `q_proj`, `k_proj`, `v_proj`, and `o_proj` together, with and without the FFN projections, but did not examine the four attention projections separately due to limited computational resources. In future work, we would like to explore these modules individually to see how they affect classification performance and whether some configurations could achieve good results with fewer trainable parameters.